

Aceleração em GPU do Cálculo Exato de Índices de Validação de Agrupamentos: uma abordagem *matrix-free* para os índices de Dunn, Silhueta e Davies-Bouldin

Henrique M. M. Miranda, Cindy S. G. Rabelo,
Luiany G. Carvalho

¹Instituto de Informática – Universidade Federal de Goiás (UFG)
Goiânia – GO – Brasil

{henrique.mendonca, cindy_stephanie, luiany}@discente.ufg.br

Resumo. A validação interna de agrupamentos (clustering) por meio de índices como Dunn, Silhueta e Davies-Bouldin exige o cálculo das distâncias euclidianas par-a-par entre todos os pontos, com custo de tempo $O(N^2)$, o que inviabiliza grandes volumes de dados em processadores sequenciais. Este trabalho apresenta uma implementação **paralela e exata** desses três índices em GPU com CUDA, adotando uma estratégia *matrix-free* que **não materializa** a matriz de distâncias $N \times N$: as distâncias são recomputadas sob demanda dentro dos kernels, reduzindo a memória de $O(N^2)$ para $O(N \cdot D)$ e viabilizando $N = 100.000$ pontos. Comparando contra um baseline em CPU (sequencial e paralelo via OpenMP) numa mesma plataforma, obtivemos speed-up de até **24,9×** (vs. CPU sequencial) e **24,5×** (vs. CPU OpenMP), com correteza validada em três camadas independentes (caso analítico, scikit-learn e equivalência $CPU \equiv GPU$).

Abstract. Internal validation of clustering through indices such as Dunn, Silhouette and Davies-Bouldin requires computing pairwise Euclidean distances among all points, with $O(N^2)$ time cost, which is prohibitive for large datasets on sequential processors. This work presents a **parallel and exact** GPU/CUDA implementation of these three indices using a *matrix-free* strategy that **does not materialize** the $N \times N$ distance matrix: distances are recomputed on demand inside the kernels, reducing memory from $O(N^2)$ to $O(N \cdot D)$ and enabling $N = 100,000$ points. Against a CPU baseline (sequential and OpenMP-parallel) on the same platform, we achieved speed-ups up to **24.9×** (vs. sequential CPU) and **24.5×** (vs. OpenMP CPU), with correctness validated in three independent layers (analytical case, scikit-learn, and $CPU \equiv GPU$ equivalence).

1. Introdução

A análise de agrupamentos (*clustering*) é uma técnica fundamental de aprendizado não supervisionado, na qual objetos são particionados em grupos de forma que elementos de um mesmo grupo sejam mais similares entre si do que em relação aos demais. Algoritmos como K-Means e DBSCAN produzem partições, mas *toda* partição precisa ser **avaliada**: é necessário medir, sem rótulos de referência, se os grupos obtidos são *coesos* internamente e *bem separados* entre si. Essa medição é feita por **índices internos de validação**,

sendo três dos mais consagrados o **Índice de Dunn** [1], o **Coeficiente de Silhueta** [2] e o **Índice de Davies-Bouldin** [3].

O gargalo computacional comum a Dunn e Silhueta é a necessidade de computar as distâncias euclidianas *par-a-par* entre todos os N pontos, resultando em complexidade de tempo $O(N^2 \cdot D)$, onde D é a dimensionalidade. Ao dobrar N , o esforço quadruplica; para dezenas ou centenas de milhares de pontos, a validação sequencial em CPU torna-se proibitiva. Essa limitação motiva o uso de arquiteturas de *paralelismo de dados massivo*, como as GPUs, nas quais milhares de *threads* executam simultaneamente a mesma operação simples (cálculo de distância) sobre dados distintos.

O artigo de referência que fundamenta este projeto, *Parallel and scalable Dunn Index for the validation of big data clusters* [4], ataca o mesmo gargalo, porém com uma estratégia distinta: distribui o cálculo em um *cluster* de máquinas usando Apache Spark e emprega **amostragem** (*Sketch and Validate*) para **aproximar** o índice em larga escala. O presente trabalho propõe uma abordagem **complementar**: em vez de aproximar de forma distribuída, calculamos os índices de forma **exata**, acelerados pelo paralelismo de *threads* de uma única GPU.

As principais contribuições deste trabalho são:

- Uma implementação CUDA **exata** dos três índices (Dunn, Silhueta e Davies-Bouldin) com reduções em memória compartilhada e operações atômicas;
- Uma estratégia **matrix-free** que evita materializar a matriz de distâncias $N \times N$, reduzindo a memória de $O(N^2)$ para $O(N \cdot D)$ e permitindo escalar a aplicação até $N = 100.000$ pontos (inviável na abordagem com matriz explícita);
- Uma avaliação experimental **justa**, comparando a GPU não apenas com a CPU sequencial, mas também com a CPU paralela (OpenMP), com *speed-up* e *break-down* de tempo por etapa;
- Uma **validação de correitude** em três camadas independentes, demonstrando que o ganho de desempenho não sacrifica a exatidão numérica.

2. Formulação do Problema

Seja $X = \{x_1, x_2, \dots, x_N\}$ um conjunto de N pontos em $\mathbb{R}^D$, particionado em K *clusters* $\{C_1, \dots, C_K\}$. A distância euclidiana entre dois pontos é $d(x_i, x_j) = \sqrt{\sum_{l=1}^D (x_{i,l} - x_{j,l})^2}$.

2.1. Índice de Dunn

O Índice de Dunn favorece partições com grupos compactos e bem separados:

$$\text{Dunn} = \frac{\min_{1 \leq a < b \leq K} \delta(C_a, C_b)}{\max_{1 \leq c \leq K} \Delta(C_c)}, \quad (1)$$

onde $\delta(C_a, C_b) = \min_{x \in C_a, y \in C_b} d(x, y)$ é a separação inter-*cluster* e $\Delta(C_c) = \max_{x, y \in C_c} d(x, y)$ é o diâmetro intra-*cluster*. Valores **maiores** indicam melhor agrupamento.

2.2. Coeficiente de Silhueta

Para cada ponto i define-se

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad (2)$$

em que $a(i)$ é a distância média de i aos demais pontos do seu próprio *cluster* e $b(i) = \min_{C \neq C_{own}} \frac{1}{|C|} \sum_{j \in C} d(x_i, x_j)$ é a menor distância média de i a um *cluster* vizinho. O índice global é $S = \frac{1}{N} \sum_{i=1}^N s(i) \in [-1, 1]$; valores **maiores** são melhores.

2.3. Índice de Davies-Bouldin

Baseado em centróides μ_c :

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \frac{S_i + S_j}{M_{ij}}, \quad (3)$$

onde $S_i = \frac{1}{|C_i|} \sum_{x \in C_i} \|x - \mu_i\|$ é a dispersão interna e $M_{ij} = \|\mu_i - \mu_j\|$ a distância entre centróides. Valores **menores** são melhores. Note que DB tem custo $O(N \cdot D + K^2 D)$, não dependendo das distâncias par-a-par.

2.4. O gargalo de complexidade

Dunn e Silhueta exigem todas as $N(N - 1)/2$ distâncias par-a-par ($\approx 5 \times 10^9$ pares distintos para $N = 100.000$), levando a um custo de tempo $O(N^2 D)$; como na formulação *matrix-free* cada distância é recomputada sob demanda, isso equivale a $\sim 10^{10}$ avaliações de distância. Além do tempo, a abordagem ingênua que *armazena* a matriz de distâncias $D \in \mathbb{R}^{N \times N}$ incorre em custo de **memória** $O(N^2)$ — proibitivo, como detalhado na Seção 4.2.

3. Objetivos

O objetivo geral é **acelerar em GPU o cálculo exato** dos três índices de validação, superando um *baseline* em CPU. Como objetivos específicos:

1. Transferir a carga quadrática do cálculo de distâncias para os milhares de núcleos da GPU, explorando a hierarquia de memória (registros, memória compartilhada e global);
2. Eliminar o gargalo de memória $O(N^2)$ por meio de uma formulação *matrix-free*, viabilizando o processamento de até 10^5 pontos;
3. Estabelecer uma comparação *justa* entre CPU sequencial, CPU paralela (OpenMP) e GPU, sob a mesma plataforma e mesmos dados;
4. Garantir a corretude numérica dos *kernels* paralelos por meio de validação multi-camada.

4. Metodologia

4.1. Baseline em CPU com OpenMP

O *baseline* implementa os três índices em C++ com o mesmo algoritmo *matrix-free*, garantindo uma comparação simétrica com a GPU. Os laços externos de Dunn e Silhueta são paralelizados com OpenMP por meio de `#pragma omp parallel for`

com cláusulas de redução (`reduction(max:...)`, `reduction(min:...)` e `reduction(+:...)`), eliminando condições de corrida. O número de *threads* é controlado pela variável de ambiente `OMP_NUM_THREADS`, permitindo medir, com o mesmo binário, tanto a execução sequencial (1 *thread*) quanto a paralela.

4.2. Estratégia *matrix-free*

A decisão de projeto central deste trabalho é **não armazenar** a matriz de distâncias. Em precisão dupla, ela ocuparia $N^2 \times 8$ bytes, conforme a Tabela 1: para $N = 50.000$ (20 GB) já excede tanto a memória da GPU utilizada (16 GB) quanto os ≈ 12 GB de RAM do ambiente, e para $N = 100.000$ (80 GB) torna-se totalmente inviável — além de ultrapassar o limite de indexação por inteiro de 32 bits em $N \cdot N$. Recomputando as distâncias sob demanda dentro dos *kernels*, a memória cai para $O(N \cdot D)$ (poucos megabytes), ao custo de recalculer distâncias — troca amplamente vantajosa, pois é o que viabiliza a escala.

Tabela 1. Custo de memória da matriz de distâncias $N \times N$ (`double`).

N	Matriz $N \times N$	Cabe na GPU (16 GB)?
8.000	0,5 GB	Sim
32.000	8,2 GB	Sim
50.000	20 GB	Não
100.000	80 GB	Não

4.3. *Kernels* CUDA

Dunn (`dunn_rowwise_kernel`). Lançamos N blocos de 256 *threads* (um bloco por ponto i). As coordenadas de i são carregadas em memória compartilhada e reutilizadas por todas as *threads* do bloco, que varrem os demais pontos em *grid-stride*, computando $d(i, j)$ sob demanda e acumulando o máximo intra-*cluster* e o mínimo inter-*cluster* locais. Em seguida, uma **redução em árvore** em memória compartilhada ($\log_2 256 = 8$ passos, sincronizados por `__syncthreads()`) consolida o resultado do bloco. A CPU executa apenas a redução global final, de custo $O(N)$, sobre dois vetores de tamanho N — evitando transferir a matriz completa.

Silhueta (`silhouette_rowwise_kernel`). Também um bloco por ponto, usando **memória compartilhada dinâmica** de tamanho $256 \times K$ para acumular, por *cluster*, a soma das distâncias de i aos demais pontos. Após uma redução por *cluster*, a *thread* 0 calcula $a(i)$, $b(i)$ e $s(i)$.

Davies-Bouldin. Como não depende de distâncias par-a-par, é implementado por uma cadeia de *kernels* de custo $O(N)$: acumulação de centróides e dispersões via `atomicAdd` (com *fallback* via *compare-and-swap* para `double` em arquiteturas anteriores a `sm_60`), seguida do cálculo das razões R_{ij} em paralelo por *cluster*.

4.4. Precisão configurável

O código suporta `double` (padrão, validado) e `float` (compilando com `-DUSE_FLOAT`). Como a GPU utilizada é significativamente mais lenta em precisão dupla, o modo `float` oferece um *trade-off* entre velocidade e exatidão; os somatórios/reduções acumulam sempre em `double` para preservar a precisão.

4.5. Validação de corretude

A corretude é tratada como *pré-condição* e verificada em três camadas: (i) um **caso analítico** do Dunn com 4 pontos de valor conhecido ($Dunn = 2,0$); (ii) comparação de Silhueta e DB com a biblioteca de referência **scikit-learn** [5]; e (iii) **equivalência CPU $\equiv$ GPU** em todos os tamanhos. O *script* de benchmark aborta a execução caso qualquer camada diverja além de 10^{-5} .

4.6. Ambiente experimental

Os dados são sintéticos, gerados com `make_blobs` ($D = 4$, $K = 5$, semente fixa) para garantir reprodutibilidade. Os experimentos foram executados no Google Colab com GPU **NVIDIA Tesla T4** (16 GB) e CPU de 2 *vCPUs*; CPU e GPU compartilham a mesma máquina, tornando a comparação de *speed-up* justa. Cada medição é a **média de múltiplas repetições** (5 para $N \leq 8000$, 3 para $N \leq 32000$ e 2 para os maiores); o desvio-padrão correspondente acompanha o código-fonte e é usado na discussão de significância. Compilação: `g++ -O3 -fopenmp` (CPU) e `nvcc -O3` (GPU). O código completo está disponível publicamente¹.

5. Resultados e Discussão

5.1. Corretude

Todas as três camadas de validação passaram. O caso analítico do Dunn resultou em 2,000000 tanto na CPU quanto na GPU (erro 0). A comparação com o scikit-learn ($N = 150$) apresentou diferença residual de $\sim 2,3 \times 10^{-9}$ (Silhueta) e $\sim 3,2 \times 10^{-9}$ (DB), no limite da precisão de ponto flutuante. A equivalência CPU $\equiv$ GPU foi de **100%** em todos os tamanhos, confirmando que a aceleração não compromete a exatidão.

5.2. Tempo de execução e *speed-up*

A Tabela 2 apresenta os tempos médios e o *speed-up*. A GPU processa $N = 100.000$ pontos em **3,24 s**, contra **80,6 s** da CPU sequencial e **79,2 s** da CPU paralela (OpenMP), resultando em *speed-up* de **24,9 $\times$** e **24,5 $\times$** , respectivamente.

A Figura 1 mostra os tempos em escala linear (à esquerda, evidenciando o tamanho do ganho) e log-log (à direita): as três curvas exibem inclinação ≈ 2 , confirmando o comportamento $O(N^2)$, com a curva da GPU deslocada para baixo — a distância vertical corresponde ao *speed-up*.

A Figura 2 apresenta o *speed-up* em função de N (eixo logarítmico). O ganho **crece monotonicamente** com o tamanho do problema: em N pequeno, o custo fixo (inicialização e transferências) domina e o *speed-up* é modesto; à medida que o trabalho aumenta, esse custo é amortizado. A anomalia em $N = 250$ na curva OpenMP deve-se ao *overhead* de criação de *threads* em problemas muito pequenos.

5.3. Decomposição do tempo de GPU (*breakdown*)

A Figura 3 decompõe o tempo de GPU nas **quatro etapas instrumentadas** (H2D, Dunn, Silhueta e Davies-Bouldin). Dunn e Silhueta — as métricas que dependem de distâncias

¹<https://github.com/Ricktheus/Trabalho-final-Cuda>

Tabela 2. Tempo total (s) e *speed-up* por tamanho do conjunto. SU_1 = GPU vs. CPU 1-thread; SU_Ω = GPU vs. CPU OpenMP.

N	CPU-1 (s)	CPU-OMP (s)	GPU (s)	SU_1	SU_Ω	Corretude
250	0,0018	0,0077	0,0011	1,6×	6,8×	100%
500	0,0029	0,0029	0,0021	1,4×	1,4×	100%
1.000	0,0090	0,0087	0,0026	3,4×	3,3×	100%
2.000	0,0365	0,0326	0,0055	6,6×	5,9×	100%
4.000	0,1215	0,1150	0,0124	9,8×	9,3×	100%
8.000	0,6943	0,4576	0,0426	16,3×	10,7×	100%
16.000	2,2145	1,8804	0,1232	18,0×	15,3×	100%
32.000	8,2827	8,1136	0,4033	20,5×	20,1×	100%
50.000	20,4357	19,4384	0,9273	22,0×	21,0×	100%
100.000	80,6082	79,2305	3,2354	24,9×	24,5×	100%

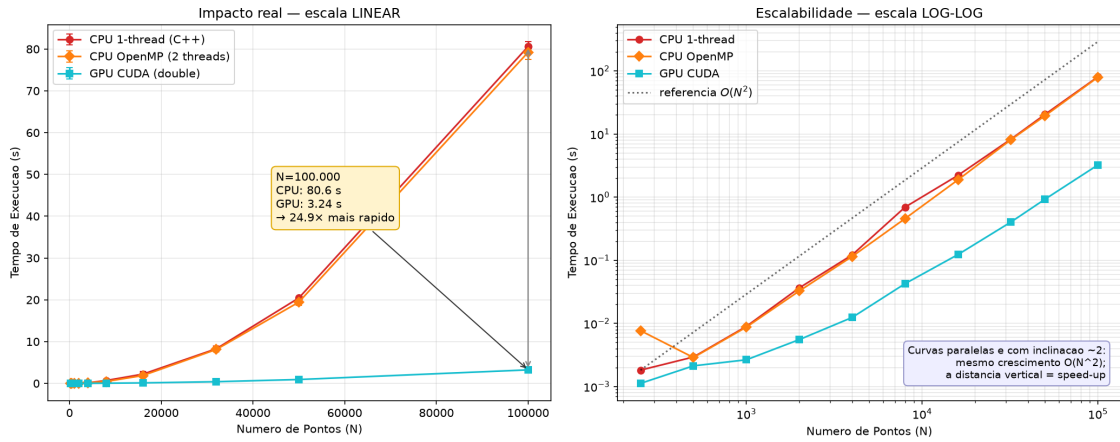


Figura 1. Tempo de execução vs. N . Esquerda: escala linear, com o *gap* em $N = 100.000$ destacado. Direita: escala log-log, com reta de referência $O(N^2)$.

par-a-par — concentram $\approx 99,95\%$ desse tempo de *kernel* (em $N = 100.000$, 1,62 s e 1,49 s, respectivamente), enquanto Davies-Bouldin e a transferência *Host-to-Device* (H2D) são desprezíveis ($< 0,001$ s). A soma das etapas ($\approx 3,11$ s) é ligeiramente inferior ao tempo total de GPU da Tabela 2 (3,24 s): a diferença de $\approx 0,12$ s ($\approx 3,7\%$) corresponde a *overhead* fora dessas etapas (alocações, *kernel* do caso analítico, cópias *Device-to-Host* e reduções no *host*). Esse resultado tem implicação prática direta: como as transferências são mínimas na formulação *matrix-free*, técnicas de sobreposição de cópia e computação (CUDA *streams*) trariam ganho irrelevante neste caso — o problema é *compute-bound*.

5.4. Discussão

Três aspectos merecem destaque. Primeiro, o *speed-up* foi medido também contra a **CPU paralela** (OpenMP), e não apenas contra um único núcleo. Cabe a ressalva de que as 2 *vCPUs* do Google Colab compartilham um mesmo núcleo físico (dois *hyperthreads*): para uma carga *compute-bound* como esta, o OpenMP quase não acelera em N grande ($80,61 \pm 1,17$ s contra $79,23 \pm 1,78$ s em $N = 100.000$, um ganho de apenas $\approx 1,02\times$, den-

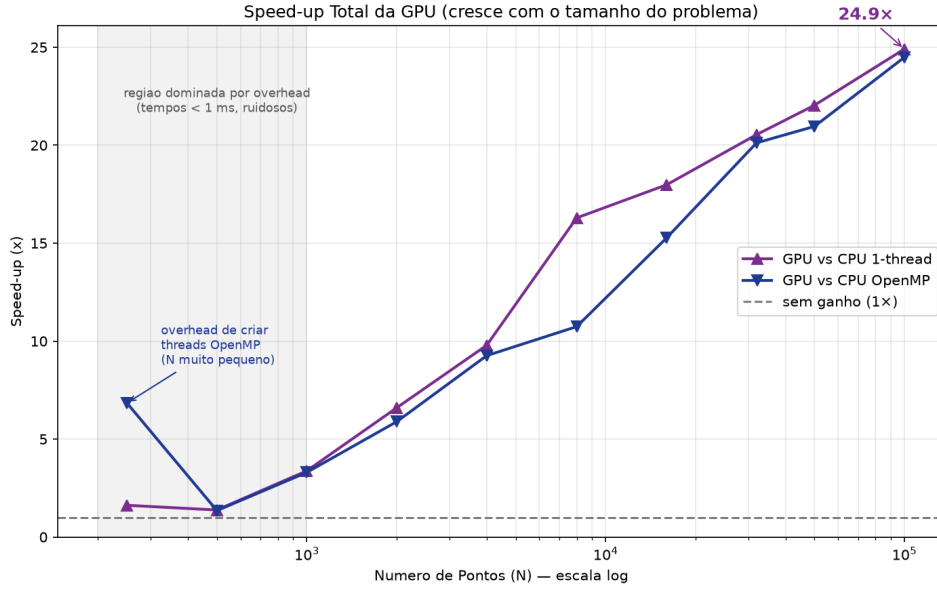


Figura 2. Speed-up da GPU em função de N (eixo N logarítmico), contra a CPU sequencial e a CPU OpenMP.

tro do desvio-padrão), o que aproxima SU_1 e SU_Ω . O resultado robusto, portanto, é que mesmo o *melhor* desempenho de CPU disponível na plataforma permanece $\approx 24\times$ atrás da GPU; aferir o *speedup* de escalabilidade forte do OpenMP exigiria uma CPU com mais núcleos físicos. Segundo, os ganhos são **conservadores**: a T4 é particularmente fraca em precisão dupla, de modo que GPUs de maior porte elevariam substancialmente o *speed-up*. Terceiro, em relação ao artigo-base [4], nossa abordagem é **exata** (sem amostragem) e executa em uma **única** GPU, sendo complementar à estratégia distribuída e aproximada baseada em Spark: para volumes na casa dos milhões, as duas ideias poderiam ser combinadas.

5.5. Limitações

Reconhecem-se as seguintes limitações: (i) por ser *matrix-free*, Dunn e Silhueta *recomputam* distâncias separadamente, o que um *kernel* fundido poderia evitar; (ii) os experimentos usam dados sintéticos bem-comportados (*blobs*), sem clusters de formato irregular ou dados reais; (iii) o teto de $N = 100.000$ é imposto pelo *tempo* da CPU de referência, não mais pela memória; (iv) o *kernel* de Silhueta usa memória compartilhada proporcional a K , limitando K muito elevados.

6. Conclusão

Este trabalho apresentou uma implementação paralela e **exata** em GPU dos índices de Dunn, Silhueta e Davies-Bouldin para validação de agrupamentos. A principal contribuição — a estratégia *matrix-free* — eliminou o gargalo de memória $O(N^2)$, reduzindo o consumo a $O(N \cdot D)$ e viabilizando o processamento de até 100.000 pontos, escala inalcançável pela abordagem com matriz explícita. Mantendo corretude exata (validada em três camadas independentes), obtivemos *speed-up* de até **24,9 $\times$** sobre a CPU sequencial e **24,5 $\times$** sobre a CPU paralela (OpenMP), com o ganho crescendo com o tamanho do problema.

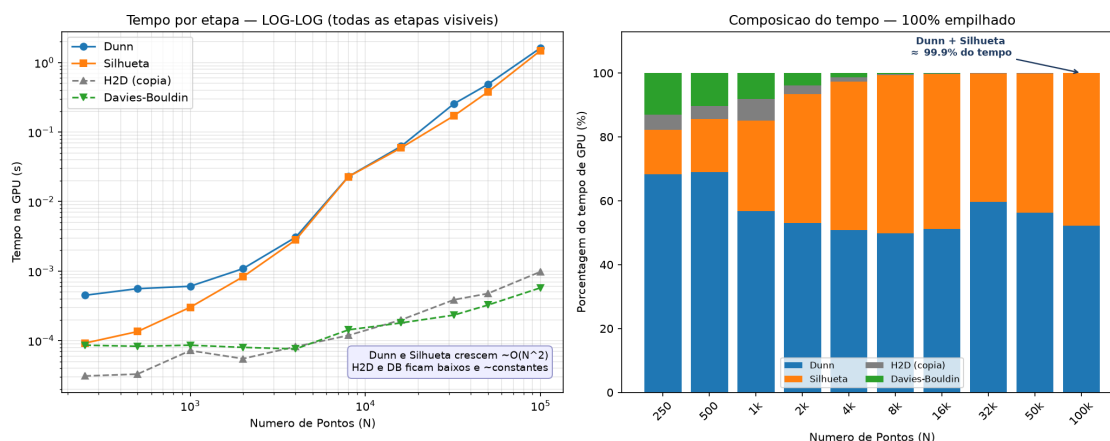


Figura 3. Decomposição do tempo de GPU por etapa. Esquerda: escala log-log (todas as etapas visíveis). Direita: composição percentual.

Como trabalhos futuros, destacam-se: a **fusão de kernels** para evitar a recomputação de distâncias; o uso de **múltiplas GPUs** e precisão mista para alcançar a casa dos milhões de pontos; a combinação com técnicas de **amostragem** [4] para validação aproximada em *big data*; e a avaliação sobre *datasets* reais e com clusters de formato arbitrário.

Referências

- [1] DUNN, J. C. Well-separated clusters and optimal fuzzy partitions. *Journal of Cybernetics*, v. 4, n. 1, p. 95–104, 1974.
- [2] ROUSSEUW, P. J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, v. 20, p. 53–65, 1987.
- [3] DAVIES, D. L.; BOULDIN, D. W. A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, v. PAMI-1, n. 2, p. 224–227, 1979.
- [4] NCIR, C. E. B.; HAMZA, A.; BOUAGUEL, W. Parallel and scalable Dunn Index for the validation of big data clusters. *Parallel Computing*, v. 102, 102751, 2021.
- [5] PEDREGOSA, F. *et al.* Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- [6] NVIDIA. *CUDA C++ Programming Guide*. NVIDIA Corporation, 2024.
- [7] OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface, Version 5.2*. 2021.